

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 算法设计

项目名称： 递归与分治

2023年秋季学期

上机时间：第 1 周 星期 2 第 3 ~ 4 节

学生姓名： 张清晨 专业班级： 计算机 2101

指导教师： 刘俊岭 成 绩：

项目名称	递归与分治		学时	4
日期：2023 年 8 月 29 日		第 1 教学周		
硬件及软件				
名 称		版本、型号		
Visual C++		6.0 及以上		
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度。 本次任务： 实验 2-1 全排列问题 实验 2-2 大整数乘法问题 实验 2-3 棋盘覆盖问题				

实验 2-1 全排列问题

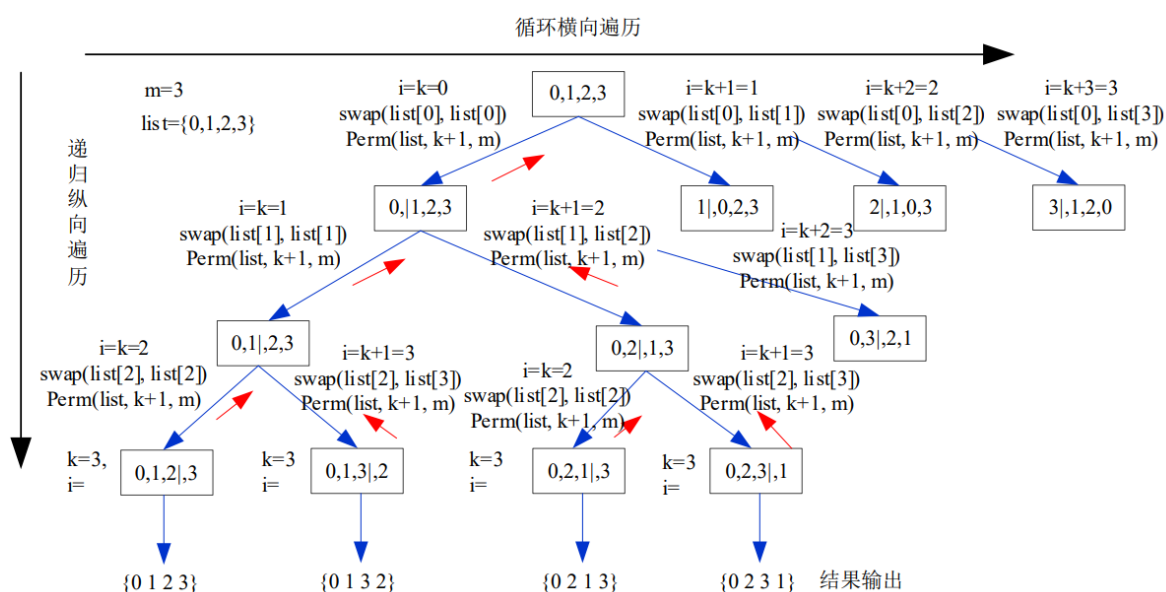
1. 问题描述:

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \dots, r_n\}$ 的全排列。设 $R=\{r_1, r_2, \dots, r_n\}$ 是要进行排列的 n 个元素， $R_i=R-\{r_i\}$ 。集合 X 中元素的全排列记为 $\text{perm}(X)$ 。 $(r_i)\text{perm}(X)$ 表示在全排列 $\text{perm}(X)$ 的每一个排列前加上前缀 r_i 得到的排列。

R 的全排列可归纳定义如下:

- 当 $n=1$ 时, $\text{perm}(R)=(r)$, 其中 r 是集合 R 中唯一的元素;
- 当 $n>1$ 时, $\text{perm}(R)$ 由 $(r_1)\text{perm}(R_1)$, $(r_2)\text{perm}(R_2)$, $\dots$, $(r_n)\text{perm}(R_n)$ 构成。

2. 算法实例:



3. 程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
```

```
using namespace std;
```

```
void perm(int list[], int k, int m);
void swap(int& a, int& b);
```

```

int main()
{
    int a[5] = {0,1,2,3,4};
    perm(a, 0, 3);
    printf("\n");

    system("PAUSE");
    return 0;
}
void perm(int list[], int k, int m)
{
    if (k == m)
    {
        //只剩下一个元素
        for (int i = 0; i <= m; i++)
            cout << list[i];
        cout << endl;
    }
    else
        for (int i = k; i <= m; i++)
        {
            swap(list[i], list[k]); //将第 i 个元素与第 k 个元素交换位置
            perm(list, k + 1, m); //递归排列第 k+1 到第 m 个元素
            swap(list[i], list[k]); //第 i 和第 k 个交换元素的重新换回正常位置
        }
}
void swap(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}

```

4. 程序验证:

```

int a[5] = {0,1,2,3,4};
perm(a, 0, 3);
printf("\n");

```

```
E:\算法设计与分析\1\实验2-1 全排列问题.exe
0123
0132
0213
0231
0321
0312
1023
1032
1203
1230
1320
1302
2103
2130
2013
2031
2301
2310
3120
3102
3210
3201
3021
3012
请按任意键继续. . .
```

```
int a[5] = {0,1,2,3,4};
perm(a, 1, 3);
printf("\n");
```

```
E:\算法设计与分析\1\实验2-1 全排列问题.exe
0123
0132
0213
0231
0321
0312
请按任意键继续. . .
```

5. 算法分析:

该算法是递归算法。递归算法就是把问题分解成规模更小、具有与原来问题相同解法的问题。

递归纵向遍历 n 次，循环横向遍历第一层有 n 次，第二层有 n 个 $n-1$ 次，下面的每一层依次递减：

时间复杂度： $O(n*n!)$

空间复杂度： $O(n)$

实验 2-2 大整数乘法问题

1. 问题描述:

分治法求两个 N 位的整数的乘积。

2. 算法实例:

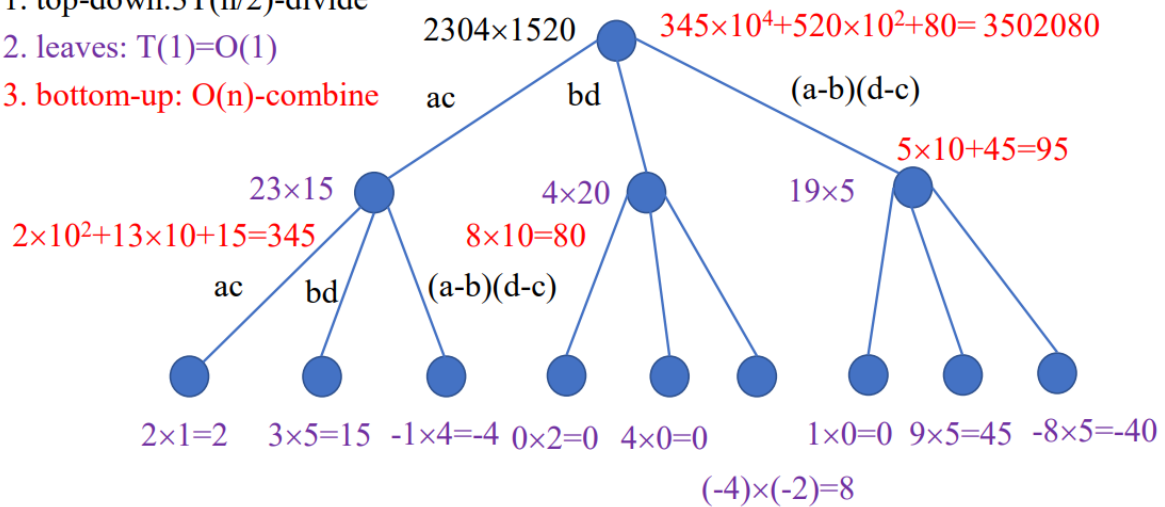
$$XY = ac \cdot 10^n + ((a-b)(d-c) + ac + bd) \cdot 10^{n/2} + bd$$

采用分治法计算 2304×1520 ，在递归树的结点处依次标注子问题及其解

1. top-down: $3T(n/2)$ -divide

2. leaves: $T(1)=O(1)$

3. bottom-up: $O(n)$ -combine



3. 程序代码，完整可运行程序，并加注释。

```
#include <math.h>
#include <tchar.h>
#include <iostream>
using namespace std;

int IntegerMultiply(int x, int y, int N)
{
    if ((0 == x) || (0 == y))
        return 0;
    if (1 == N)
        return x*y;
    else
    {
        int a = x / (int)pow(10., (int)N / 2);
        int b = x - a * (int)pow(10., N / 2);
        int c = y / (int)pow(10., (int)N / 2);
        int d = y - c * (int)pow(10., N / 2);
```

```

        int ac = IntegerMultiply(a, c, N/2);
        int bd = IntegerMultiply(b, d, N/2);
        int ambdmc = IntegerMultiply(a-b, d-c, N/2) + ac + bd;
        return (ac * (int)pow(10., N) + ambdmc * (int)pow(10., N / 2) + bd);
    }
}

int _tmain(int argc, _TCHAR* argv[])
{
    int x = 2304;
    int y = 1520;

    cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
    cout << "x * y = " << x*y << endl;
    system("pause");
    return 0;
}

```

4. 程序验证:

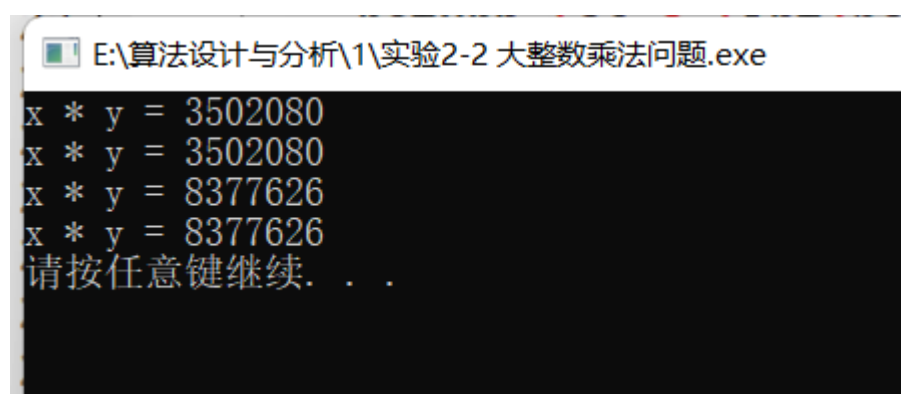
```

int x = 2304;
int y = 1520;

cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
cout << "x * y = " << x*y << endl;

x = 1234;
y = 6789;
cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
cout << "x * y = " << x*y << endl;

```



5. 算法分析, 时间复杂度。

当 $n=1$ 时时, 间复杂度为 $O(1)$.

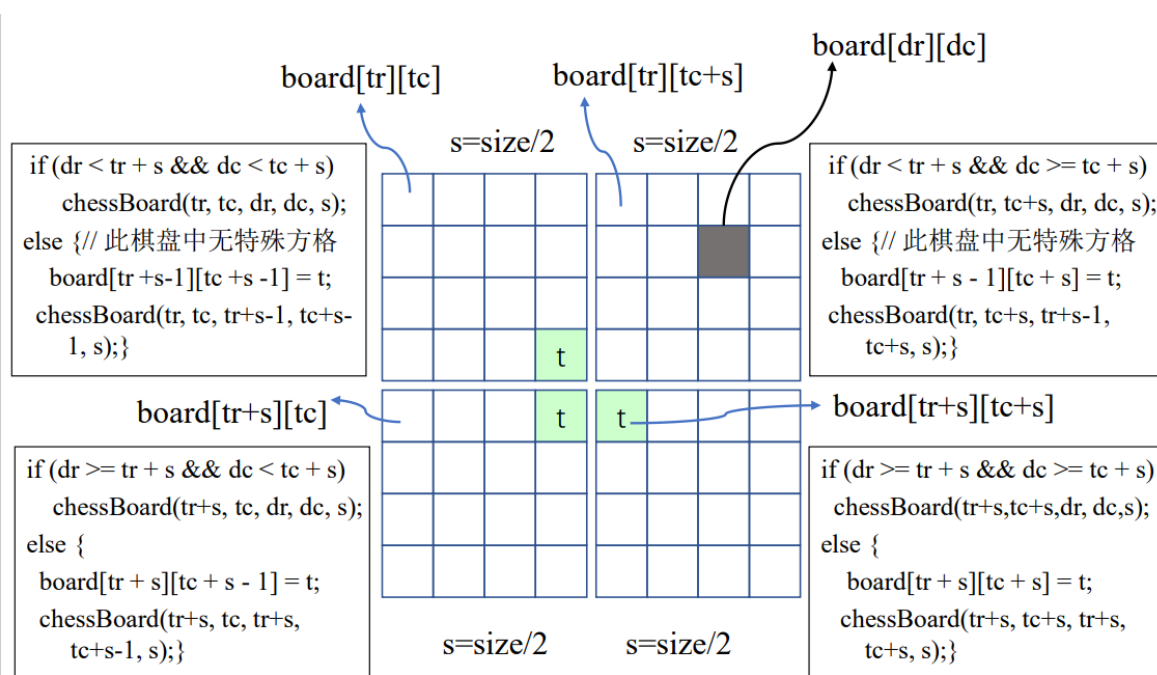
当 $n>1$ 时, $T(n) = 3T(n/2) + O(n)$, 时间复杂度: $O(n^{1.59})$

实验 2-3 棋盘覆盖问题

1. 问题描述:

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称为特殊方格或缺陷方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的 4 种不同形态的 L 型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何 2 个 L 型骨牌不得重叠覆盖。在 $2^k \times 2^k$ 的棋盘覆盖中，用到的 L 型骨牌数恰为 $(4k-1)/3$, 即 $(\text{所有方格个数} - \text{特殊方格个数}) / 3$ 。

2. 算法实例:



3. 程序代码，完整可运行程序，并加注释。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>

#include <iomanip>
//setw(n)是 c++中在输出操作中使用的字段宽度设置，n 表示字段宽度。
using namespace std;
```



```

int matrix[100][100];
int num = 0;
void chessBoard(int x, int y, int a, int b, int length);

int main()
{
    int a, b, length;
    cout << "请输入棋盘的行列号";
    cin >> length;
    cout << "请输入特殊方格的行列号";
    cin >> a >> b;
    matrix[a][b] = -1;

    chessBoard(1, 1, a, b, length);

    for (int i = 1; i <= length; i++){
        for (int j = 1; j <= length; j++){
            cout << setw(4) << matrix[i][j];
        }
        cout << endl;
    }

    system("PAUSE");

    return 0;
}

void chessBoard(int x, int y, int a, int b, int length) {
    //如果棋盘上只有一个方格，且该方格为一特殊方格
    if (length == 1){
        return;
    }
    int h = length / 2;    //分割棋盘
    // int t = ++num;        //L 型骨牌号(从 1 开始)
    int t = num++;         //L 型骨牌号(从 0 开始)
    /*左上角*/
    if (a < x + h && b < y + h){    //特殊方格在此棋盘中则划分
        chessBoard(x, y, a, b, h);
    }
    else{    //否则先覆盖右下角的方格(用 t 号 L 型骨牌)，再划分
        matrix[x + h - 1][y + h - 1] = t;
        chessBoard(x, y, x + h - 1, y + h - 1, h);
    }
}

```

```

/*右上角*/
if (a < x + h && b >= y + h){    //特殊方格在此棋盘中则划分
    chessBoard(x, y + h, a, b, h);
}
else{    //否则先覆盖左下角的方格(用 t 号 L 型骨牌), 再划分
    matrix[x + h - 1][y + h] = t;
    chessBoard(x, y + h, x + h - 1, y + h, h);
}

/*右下角*/
if (a >= x + h && b >= y + h){    //特殊方格在此棋盘中则划分
    chessBoard(x + h, y + h, a, b, h);
}
else{    //否则先覆盖左上角的方格(用 t 号 L 型骨牌), 再划分
    matrix[x + h][y + h] = t;
    chessBoard(x + h, y + h, x + h, y + h, h);
}

/*左下角*/
if (a >= x + h && b < y + h){    //特殊方格在此棋盘中则划分
    chessBoard(x + h, y, a, b, h);
}
else{    //否则先覆盖右上角的方格(用 t 号 L 型骨牌), 再划分
    matrix[x + h][y + h - 1] = t;
    chessBoard(x + h, y, x + h, y + h - 1, h);
}
}

```

4.程序验证:

```

E:\算法设计与分析\1\实验2-3 棋盘覆盖问题.exe
请输入棋盘的行列号8
请输入特殊方格的行列号4 5
2 2 3 3 7 7 8 8
2 1 1 3 7 6 6 8
5 1 4 4 10 10 6 9
5 5 4 0 -1 10 9 9
17 17 18 0 0 12 13 13
17 16 18 18 12 12 11 13
20 16 16 19 15 11 11 14
20 20 19 19 15 15 14 14
请按任意键继续. . .

```

E:\算法设计与分析\1\实验2-3 棋盘覆盖问题.exe

请输入棋盘的行列号16

请输入特殊方格的行列号5 2

3	3	4	4	8	8	9	9	24	24	25	25	29	29	30	30
3	2	2	4	8	7	7	9	24	23	23	25	29	28	28	30
6	2	5	5	11	11	7	10	27	23	26	26	32	32	28	31
6	6	5	1	1	11	10	10	27	27	26	22	22	32	31	31
18	-1	19	19	1	13	14	14	39	39	40	40	22	34	35	35
18	18	17	19	13	13	12	14	39	38	38	40	34	34	33	35
21	17	17	20	16	12	12	15	42	42	38	41	37	33	33	36
21	21	20	20	16	16	15	15	0	42	41	41	37	37	36	36
66	66	67	67	71	71	72	0	0	45	46	46	50	50	51	51
66	65	65	67	71	70	72	72	45	45	44	46	50	49	49	51
69	65	68	68	74	70	70	73	48	44	44	47	53	53	49	52
69	69	68	64	74	74	73	73	48	48	47	47	43	53	52	52
81	81	82	64	64	76	77	77	60	60	61	43	43	55	56	56
81	80	82	82	76	76	75	77	60	59	61	61	55	55	54	56
84	80	80	83	79	75	75	78	63	59	59	62	58	54	54	57
84	84	83	83	79	79	78	78	63	63	62	62	58	58	57	57

请按任意键继续. . .

5.算法分析，时间复杂度。

给定 $2^k \times 2^k$ 棋盘

当 $k=0$ 时， $T(k) = O(1)$

当 $k>0$ 时， $T(k) = 4T(k-1) + O(1)$

所以，时间复杂度： $O(4^k)$

结果分析和总结：

今天是算法课的第一次上机，学习了递归和分治的思想，并用三个实例进行了算法的练习，使我收获颇多，但是对于时间复杂度的计算对我来说还是一个难点，我会加油去搞明白。递归是直接或间接调用函数自身的算法，分治是将问题分解成多个更小的问题，同时具有与原问题一样的解法。第一次学习算法课，了解了算法的基本思想，后续的算法也会认真学习。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：